Homework 2: Forward Kinematics

Fall 2026 | Due September 23, 2026 at 11:59 PM

1. Objectives

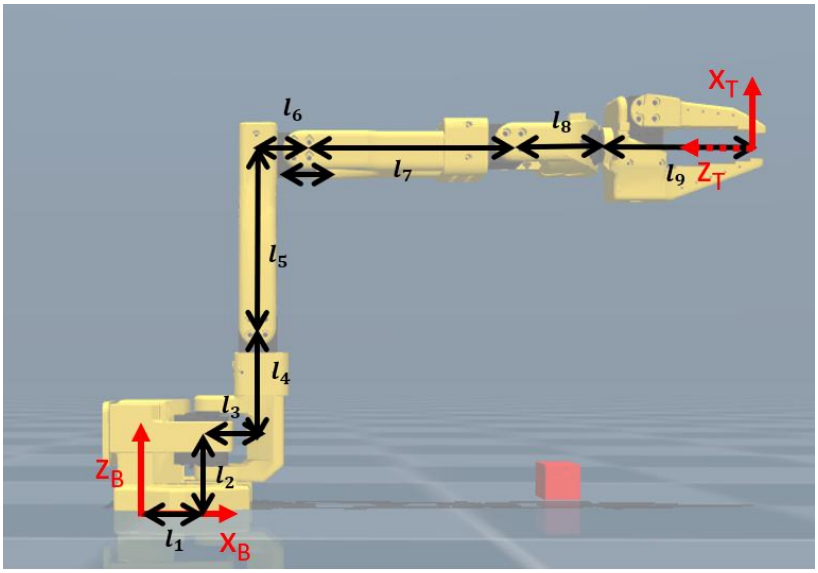
After completing this assignment, you should be able to:

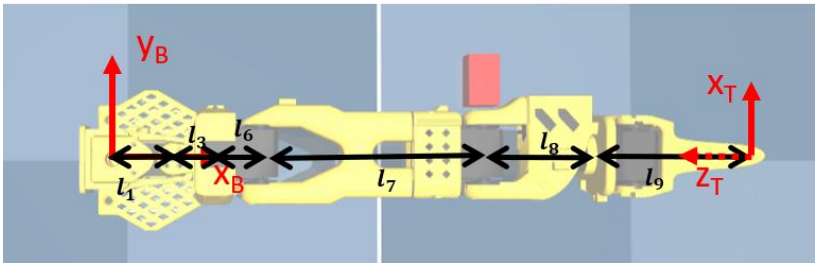
- Assign coordinate frames to a serial manipulator using the standard DH convention.
- Create a complete DH parameter table and the corresponding homogeneous transforms.
- Implement forward kinematics in Python.
- Validate position and orientation against an independent MuJoCo reference pose.

2. Files Provided

File	Purpose
student_fk.py	The only Python file you edit and submit.
test_single_fk.py	Runs your FK for one configuration and compares it with the MuJoCo oracle.
mujoco_oracle.py	Provides the reference pose from the MuJoCo robot model. Do not edit.
robot-assets/scene.xml	Loads the SO-101 model used by the oracle.

3. Robot Model and Given Dimensions





You are to determine the forward kinematics for SO0101 5 degree-of-freedom robot manipulator. A picture is shown to the right, and a screen capture from the Mujoco simulation is shown below. A version is included on the last page that is faded and might be easier for you to print and draw on if you want. The link lengths are labeled and provided in the given table

Param.	Value (m)	Description
l_1	0.0388	Base horiz. offset
l_2	0.0624	Base vert. offset
l_3	0.0304	Shoulder length
l_4	0.0542	Shoulder height
l_5	0.1126	Upper arm length
l_6	0.0280	Upper arm offset
l_7	0.1349	Forearm length
l_8	0.0611	Wrist length
l_9	0.1004	Gripper length

4. Task 1: Derive the Forward Kinematics (35 points)

Determine the transform from the given base frame to the given tool frame using the standard DH convention.

1. Draw and label every DH frame on the supplied robot figures. Clearly show each x-axis and z-axis.
2. Create a DH table with one row for each transform. Use the column order: a, α, d, θ.
3. Write the homogeneous transform for each row of your DH table.
4. Write the ordered matrix product that gives the final base-to-tool transform.

Base-frame note: The base frame shown in the figure does not coincide with the first DH frame. An additional fixed transform will be required before the DH chain. The easiest way is to consider F_B to be F_{-1}, then follow the DH convention for F_0 with respect to F_{-1}.

Tool-frame note: Your final DH frame must produce the tool-frame position and orientation shown in the supplied figure. Depending on your frame assignment, a final fixed transform may be required. DH frame assignments are not unique, but the final base-to-tool pose must be correct.

5. Task 2: Implement the FK Functions (15 points)

Open student_fk.py. Implement both required functions without changing their names or input/output formats.

5.1 dh_transform(theta, d, a, alpha)

This function receives one row of standard DH parameters and returns a 4 × 4 homogeneous transform:

```
T = Rz(theta) @ Tz(d) @ Tx(a) @ Rx(alpha)
```

Required return value:

```
T.shape == (4, 4)
```

5.2 forward_kinematics(q)

The input q is a dictionary containing five joint angles in radians:

```
q = {
    "shoulder_pan":  q1,
    "shoulder_lift": q2,
    "elbow_flex":    q3,
    "wrist_flex":    q4,
    "wrist_roll":    q5,
}
```

Your function must return exactly two NumPy arrays:

```
p, R = forward_kinematics(q)

p.shape == (3,)
R.shape == (3, 3)
```

- `p` is the tool-frame position expressed in the given base frame, in meters.
- `R` is the tool-frame rotation matrix expressed in the given base frame.
- Use radians throughout your code.
- Use your DH table and homogeneous transforms. Do not hard-code answers for individual test configurations.

6. Test One Configuration

The test script compares your result with the pose computed from the MuJoCo robot model. This MuJoCo result is the *oracle*, meaning it is the reference answer used to check your FK implementation.

1. Open test_single_fk.py.
2. Edit the five values in JOINT_CONFIGURATION. All values are in radians and must remain within the robot joint limits.
3. From the `lerobot-mujoco` directory, run the command for your operating system:

   Windows / Linux:
   ```
   uv run python robo-code/HW_02/test_single_fk.py --xml robot-assets/scene.xml
   ```

   macOS:
   ```
   uv run mjpython robo-code/HW_02/test_single_fk.py --xml robot-assets/scene.xml
   ```

   The text in red will need to be changed depending on where you downloaded the code.
4. Compare the oracle and student results. The script reports position error in meters and a rotation error based on the relative rotation matrix.
5. Repeat with several configurations, including the zero configuration and configurations that move one joint at a time.

Expected nonzero error: The SO-101 does not provide a forward-kinematics API. Instead, we use the MuJoCo model as an oracle. Given a set of joint angles, the oracle returns the pose of the robot's static finger. Your forward kinematics implementation is compared against this pose. Because the static finger is not located exactly at the idealized tool frame shown in the figure, even a correct solution may produce small non-zero errors when compared to the oracle.

Error Metrics: The position error is the Euclidean distance between the position returned by your forward kinematics function and the position returned by the oracle:

$$e_p = \| \mathbf{p}_{oracle} - \mathbf{p}_{FK} \|.$$

The rotation error is

$$e_R = \text{trace}(I_3 - \boldsymbol{R}_{oracle}^T \boldsymbol{R}_{FK})$$

A value of $e_R = 0$ indicates perfect agreement between the two orientations (a good exam question might be why this is a good rotation error metric :^))

Expected nonzero error: The SO-101 does not provide a forward-kinematics API. Instead, we use the MuJoCo model as an oracle. Given a set of joint angles, the oracle returns the pose of the robot's static finger. Your forward kinematics implementation is compared against this pose. Because the static finger is not located exactly at the idealized tool frame shown in the figure, even a correct solution may produce small non-zero errors when compared to the oracle.

7. Submission Requirements

Submit exactly these two files:

- <last_name>_<first_name>_HW2.pdf
- student_fk.py

8. Grading and Validation

The TA will evaluate student_fk.py over 100 valid robot configurations and review the code for consistency with your paper derivations. Both position and orientation will be checked.

AI Use Disclaimer: I used MS CoPilot (GPT-5) to help generate the code provided to you and make it clear. I also used it to make this assignment clearer and more detailed.